Attention 加速器 IP 设计与验证

汇报人：张仕涵

1.项目背景和公式推导

1.1项目背景

这次实践项目的题目是把 Attention 算法的一小部分写成真正能综合的硬件。Attention 本身做的事并不复杂：当前这个 Token 拿着自己的 Q 去和历史上每个 Token 的 K 比对，看它们有多像，像得越多，就从对应的 V 里取越多内容出来。

1.2 核心公式推导

$$score_j = Q \cdot K_j = \Sigma\, Q[k] \times K_j[k]$$

$$weight_j = \exp(score_j) \qquad L = \Sigma\, weight_j$$

$$ACC = \Sigma\, weight_j \times V_j \qquad O = ACC / L$$

```
module attention_ip(
    input clk, rst_n, start,
    input signed [7:0] q_in, k_in, v_in,
    input q_valid, kv_valid,
    output data_ready,
    output reg signed [7:0] out_data,
    output reg out_valid, done
);

//FSM definition
parameter IDLE = 4'd0;
parameter LOAD_Q =4'd1;
parameter LOAD_KV = 4'd2;
parameter DOT = 4'd3;
parameter EXP = 4'd4;
parameter UPDATE = 4'd5;
parameter NORM =4'd6;
parameter OUTPUT = 4'd7;

reg signed [7:0] q_buf [0:3];
reg signed [7:0] k_buf [0:7][0:3];
reg signed [7:0] v_buf [0:7][0:3];
reg signed [17:0] score_acc;
reg signed [17:0] score_buf [0:7];
```

```
exp_lut u_exp_lut(
    .x(exp_x),
    .weight(exp_weight)
);
```

顶层模块

```
LOAD_Q:
begin
    if(q_valid)
    begin
        q_buf[q_cnt] <= q_in;

        if(q_cnt == 2'd3)
        begin
            q_cnt <=2'd0;
            token_cnt <=3'b0;
            dim_cnt <=2'd0;

            state <= LOAD_KV;
        end
        else
        begin
            q_cnt <= q_cnt + 1;
        end
    end
end
```

```
LOAD_KV:
begin
    if (kv_valid)
    begin
        k_buf[token_cnt][dim_cnt] <= k_in;
        v_buf[token_cnt][dim_cnt] <= v_in;

        if (dim_cnt == 2'd3)
        begin
            dim_cnt <= 2'd0;

            if (token_cnt == 3'd7)
            begin
                token_cnt <= 3'd0;
                state <= DOT;
            end

            else
            begin
                token_cnt <= token_cnt +1'b1;
            end
        end

        else
        begin
            dim_cnt <= dim_cnt +1'b1;
        end
    end
end
```

Q,K,V的输入

```
DOT:
begin
    if (dim_cnt == 2'd3)
    begin
        score_buf[token_cnt] <= score_acc + q_buf[dim_cnt] * k_buf[token_cnt][dim_cnt];
        score_acc <= 18'sd0;
        dim_cnt <= 2'd0;
        if (token_cnt == 3'd7)
        begin
            token_cnt <= 3'd0;
            max_cnt <= 3'd0;
            exp_phase <= 1'b0;
            max_score <= score_buf[0];
            dim_cnt<= 3'd0;
            state <= EXP;
        end
        else
        begin
            token_cnt <= token_cnt +1'b1;
        end
    end
    else
    begin
        score_acc <= score_acc + q_buf[dim_cnt] * k_buf[token_cnt][dim_cnt];
        dim_cnt <= dim_cnt +1'b1;
    end
end
```

得分阶段

```
EXP:
begin
    if(!exp_phase)
    begin
        if (max_cnt == 7)
        begin
            if(max_score < score_buf[max_cnt])
            begin
                max_score <= score_buf[max_cnt];
                exp_x <= score_buf[0] - score_buf[max_cnt];
            end
            else
            begin
                exp_x <= score_buf[0] - max_score;
            end
            max_cnt <=0;
            exp_phase <= 1'b1;
            exp_cnt <= 3'b0;
            judge_L <= 1;
        end
        else
        begin
            weight_buf[exp_cnt] <= exp_weight;
            if(exp_cnt == 3'd7)
            begin
                exp_cnt <= 3'd0;
                L <= 8'b0;
                update_cnt <= 3'b0;
                state <= UPDATE;
            end
            else
            begin
                exp_cnt <= exp_cnt + 1'b1;

                exp_x <= score_buf[exp_cnt + 1'b1] - max_score;
            end
        end
```

查找得分最大值和权重阶段

```
UPDATE:
begin
    if(judge_L)
    begin
        if(update_cnt == 3'd7)
        begin
            L <= L + weight_buf[update_cnt];
            acc_acc <= 16'sd0;
            acc_cnt <= 3'd0;
            judge_L <= 1'b0;
        end
        else
        begin
            L <= L + weight_buf[update_cnt];
            update_cnt <= update_cnt + 1'b1;
        end
    end
```

权重累加得到L

```
if(acc_cnt == 3'd7)
begin
    acc_buf[dim_cnt] <= acc_acc + $signed({1'b0, weight_buf[acc_cnt]}) * v_buf[acc_cnt][dim_cnt
    acc_acc <= 16'sd0;
    acc_cnt <=0;
    if(dim_cnt == 2'd3)
    begin
        dim_cnt <= 2'b0;
        norm_cnt <= 2'b0;
        state <= NORM;
    end
    else
    begin
        dim_cnt <= dim_cnt +1'b1;
```

计算分子acc

```
NORM:
begin
    out_valid <= 1'b1;
    if (acc_buf[norm_cnt] > 0)
    begin
        out_data <= (acc_buf[norm_cnt]  + $signed({8'b0,L})/2)/ $signed({8'b0,L});
    end
    else
    begin
    out_data <= (acc_buf[norm_cnt]  - $signed({8'b0,L})/2)/ $signed({8'b0,L});
    end
    if(norm_cnt == 2'd3)
    begin
        norm_cnt <=2'b0;
        state <= OUTPUT;
    end
    else
    begin
        norm_cnt <= norm_cnt +1'b1;
    end
end

OUTPUT:
begin
    done <= 1'b1;
    state <= IDLE;
end

default:
begin

end


        endcase
    end
end
```

归一化和输出

位宽说明

输入 Q、K、V 按题目是 8 位有符号，范围 −128 到 127。Score 通路上，两个 8 位数相乘最大是 128 × 128 = 16384，4 项累加后达到 6.5 万，而 16 位有符号最大只到 32767，明显不够，所以 score_acc、score_buf、max_score 和 exp_x 统一取 18 位，能表示正负十三万多，ACC 通路上，权重最大 16、V 的绝对值最大 128，单项乘积不超过16×128=2048，8 个 Token 累加不超过 16384 的量级，16 位有符号装得下，所以 acc_acc 和 acc_buf 取 16 位。L 是 8 个权重相加，最大 128，取 8 位无符号。输出按题目要求是 8 位有符号。

验证方法与结果

- 5.1 Golden Model

```python
import random



def exp_lut(x):
    if x == 0:
        return 16
    elif x == -1:
        return 6
    elif x == -2:
        return 2
    elif x == -3:
        return 1
    else:
        return 0
Q = [random.randint(-8, 8) for _ in range(4)]

K = []
V = []

for i in range(8):
    k = [random.randint(-8, 8) for _ in range(4)]
    v = [random.randint(-8, 8) for _ in range(4)]

    K.append(k)
    V.append(v)
```

5.2 Testbench 结构

```
    clk = 1'b0;
end

always #5 clk = ~clk;

task send_q;
    input signed [7:0] data;

    begin
        @(negedge clk);
        q_in = data;
        q_valid = 1'b1;
        @(negedge clk);
        q_valid = 1'b0;
    end

endtask

task send_kv;
    input signed [7:0] k_data;
    input signed [7:0] v_data;
    begin
        @(negedge clk);
        k_in = k_data;
        v_in = v_data;

        kv_valid = 1'b1;

        @(negedge clk);

        kv_valid = 1'b0;
    end
endtask
```

```
`timescale 1ns/1ps

module tb_exp_lut;

reg  signed [17:0] x;
wire [4:0] weight;

exp_lut u_exp_lut (
    .x(x),
    .weight(weight)
);

initial begin

    x = 18'sd0;
    #10;
    $display("x = %0d, weight = %0d", x, weight);

    x = -18'sd1;
    #10;
    $display("x = %0d, weight = %0d", x, weight);

    x = -18'sd2;
    #10;
    $display("x = %0d, weight = %0d", x, weight);

    x = -18'sd3;
    #10;
    $display("x = %0d, weight = %0d", x, weight);

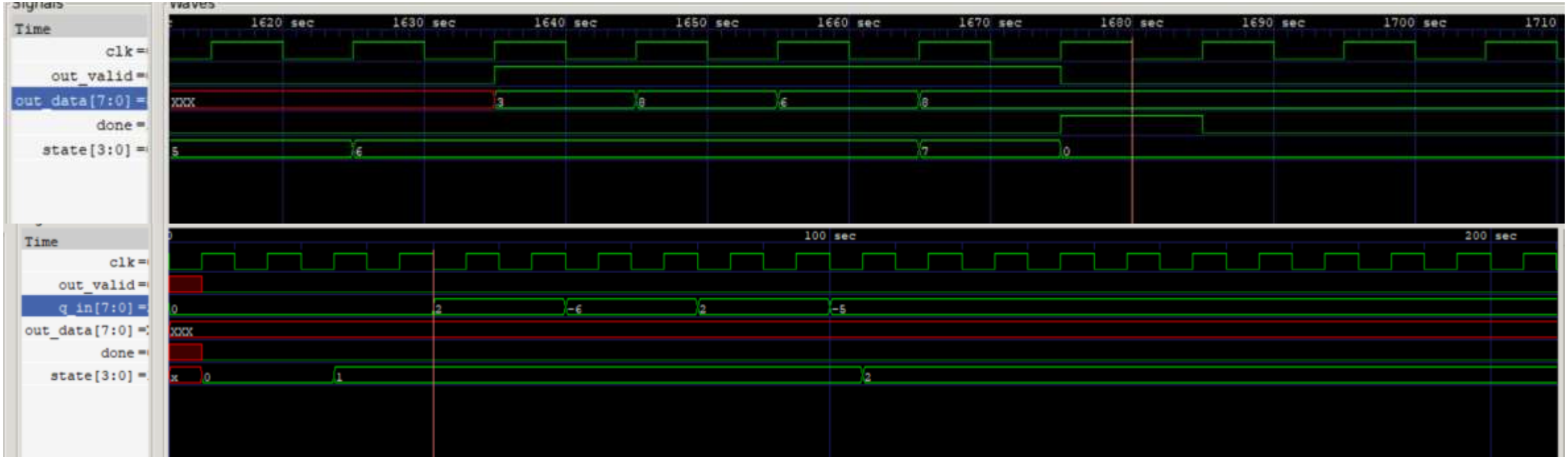
    x = -18'sd4;
    #10;
    $display("x = %0d, weight = %0d", x, weight);

    x = -18'sd100;
    #10;
    $display("x = %0d, weight = %0d", x, weight);
```

仿真波形展示

```
Score = [21, 1, 15, 39, 18, -28, -5, -12]
x_max = 39
Weight = [0, 0, 0, 16, 0, 0, 0, 0]
L = 16
ACC = [48, 128, 96, 128]
OUT = [3, 8, 6, 8]
```

```
PS E:\attention_ip> iverilog -g2012 -o sim.vvp attention_ip.v exp_lut.v tb_attention_ip.v
PS E:\attention_ip> vvp sim.vvp
VCD info: dumpfile attention.vcd opened for output.
acc_buf[0] =    48
acc_buf[1] =   128
acc_buf[2] =    96
acc_buf[3] =   128
tb_attention_ip.v:130: $finish called at 1850 (1s)
PS E:\attention_ip>
```



感谢垂听！